Revisiting Wayland for ArchLinux

Rohit Goswami · 2021-12-24 · ramblings · workflow · setup · archlinux

> *Ruminating on `wayland` and `sway` for daily use with ArchLinux*

Background

Many years ago, I found time to make an attempt to switch away from the X11 Window system to Wayland. At the time, I ended up switching back to Xorg, but I did want to revisit it. Since I returned from home recently and was gifted a Gen 6 ThinkPad X1 Carbon, I had a perfect opportunity to do so [^fn:1].

Part of this post will be interspersed with generic non-Wayland observations, but it has been a few years since my last setup and things have changed. Rather than regurgitate the fantastic ArchWiki installation guide, a FAQ style short-doc seems more appropriate.

As before, none of these configurations should be considered complete/recommended/normative; **use the installation guide** and follow the Arch Way. These are install notes, not tutorials.

ArchLinux baseline

Networking

The days of `wifi-menu` are gone. The new `iwd` interface (ArchWiki) is the newest, swankiest kid on the block [^fn:2] and works great with `NetworkManager`

Input devices

`loadkeys` for the console, `libinput` configurations can be handled via the window manager under Wayland

Audio

`pulseaudio` works well as always, though `pipewire` is pretty neat as well

Nix

Via the multi-user install

Time

Being as this is a laptop configuration, it is best to configure `chrony` (described here) as the `ntp` client and server of choice

AUR Helper

`trizen`, though I retain a soft spot for `yay`

Portability

Details are presented concisely on the ArchWiki

Device details

The ArchWiki has an entry, which includes modem setup woes and also `throttled` details for better power management (I ended up using throttled over `thermald`)

Window and login

`sway`, and `greetd`

Configurations

In general the idea is to run as few services as possible so `systemctl --type=service` is very helpful.

```
systemctl --type=service
 UNIT                                                LOAD   ACTIVE SUB     DESCRIPTION
  bolt.service                                        loaded active running Thunderbolt system service
  chronyd.service                                     loaded active running NTP client/server
  dbus.service                                        loaded active running D-Bus System Message Bus
  greetd.service                                      loaded active running Greeter daemon
  iwd.service                                         loaded active running Wireless service
  kmod-static-nodes.service                           loaded active exited  Create List of Static Device
Nodes
  lenovo_fix.service                                  loaded active running Stop Intel throttling
  lvm2-monitor.service                                loaded active exited  Monitoring of LVM2 mirrors,
snapshots etc. using dmeventd or progress polling
  NetworkManager.service                              loaded active running Network Manager
  open-fprintd.service                                loaded active running Open FPrint Daemon
...
```

rEFInd Stanzas

Some things I forgot which I should not have, and which should be taken in more of a RTFM vein.

`boot/refind_linux.conf`

for kernel parameters used with auto-detected stanzas, e.g. enabling app-armor everywhere via `lsm=landlock,lockdown,yama,apparmor,bpf`

- This **is not** where boot stanzas are to be defined

`boot/EFI/refind/refind.conf`

for manual stanzas, settings, themes and other things

Also random observations.

- `/etc/fstab` is completely useless for booting, but should mount `boot` correctly otherwise updates get messed up
- `cat /proc/cmdline` will dump the current kernel parameters
- `root=UUID=BLAH` or `root=PARTUUID=BLAH` is meant to be the **arch** root, not the **boot** partition
  - `volume` is the **boot** partition and should be set to `root=`

Network and Time

Networking with `iwd` is very well described on the ArchWiki, with the exception of its interaction with NetworkManager which essentially involves configuring it to use the IWD backend via `/etc/NetworkManager/NetworkManager.conf`:

```
[device]
wifi.backend=iwd
```

Now all that remains is to disable `wpa` and friends.

```
systemctl stop wpa_supplicant
systemctl disable wpa_supplicant
systemctl start iwd
systemctl start NetworkManager
```

Personally I combine this the NetworkManager dispatcher script for `chrony`.

Eduroam

For the most part, auto configuration works. However, for `eduroam` I needed to setup `/var/lib/iwd/eduroam.8021x` with the following:

```
[Security]
EAP-Method=TTLS
EAP-Identity=$BLAH@DOMAIN
EAP-TTLS-Phase2-Method=MSCHAPV2
EAP-TTLS-Phase2-Identity=$BLAH@DOMAIN

[Settings]
Autoconnect=true
```

With this `iwctl` works well enough:

```
station wlan0 connect eduroam
```

Mobile broadband

The Fibocom L850-GL hardware can be setup via the experimental native xmm7360-pci driver. Unfortunately, upstream has not processed a couple of important pull requests. So using my fork:

```
git clone git@github.com:HaoZeke/xmm7360-pci.git
cd xmm7360-pci
sudo /usr/bin/pip install --user pyroute2 ConfigArgParse
make && make load
make && make load # first one doesn't take
sudo /usr/bin/python rpc/open_xdatachannel.py --apn net.nova.is # or whatever your apn is
```

To test that everything is working, a new device should have shown up with `ip addr`.

```
6: wwan0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen
1000
    link/none
```

It should have some `inet` addresses as well. Unfortunately, the steps above need to be run every time, so whenever needed:

```
make && make load
make && make load # first one doesn't take
sudo /usr/bin/python rpc/open_xdatachannel.py --apn net.nova.is
```

Audio

As discussed earlier, no special requirements need to be considered beyond setting up `/etc/libao.conf` which in itself is only needed for pianobar.

```
default_drive=pulse
# dev=default
quiet
```

Actually this also holds true for `pipewire` and that's pretty much what I decided to go with this time around.

Snap and AppArmor

In general, it seems that `snap` packages have better support for `wayland`, like for Telegram. Setting this up involves working out `apparmor` as well via the kernel parameters. However, for me and some other people apparently, enabling `apparmor.service` corresponds to taking a few seconds of added boot time. Unacceptable for me at any rate. So no `snapd` either.

Flatpak uses namespaces, but honestly on an ArchLinux machine it is always better to just build things.

Intel graphics

Following along the standard wiki page, I ended up setting `fastboot` and `enable_fbc`, verified with:

```
sudo systool -m i915 -av
```

The settings can be passed through a kernel parameter or more simply via:

```
# sudoedit /etc/modprobe.d/i915.conf
options i915 enable_fbc=1 fastboot=1 intel_iommu=on,igfx_off
```

Window / Session Management

Window and session managers are not exactly a dime a dozen. The X window system has served me well for many many years yet, for this post, of the X window system, the less said the better. Wayland on the other hand, is fast, shiny, and doesn't as yet have 233 configuration flags.

Sway basics

`Weston` remains the reference implementation, and GNOME has continued championing the cause of Wayland which is great. However, since my foray in 2018; tiling window managers have come a long way. Migrating away from the MacOS' "tiling managers" was smooth and very reminsicent of my older `i3-gaps` setup.

High DPI

An interesting quirk of `sway` is that Hi-DPI support is still at the draft stage. For the most part this is becoming a non-issue as more applications support Wayland out of the box, or at-least optionally.

However, certain applications are unusable without the HiDPI support patches, including most `java` packages like `cryptomator`. Thus it is best to grab the modified versions and configure them.

```
trizen -S sway-hidpi-git wlroots-hidpi-git xorg-xwayland-hidpi-git
```

Note that by default `sway` supports fractional scaling too via, `swaymsg output $MON scale 1.5` and also when in the configuration. However, the patches are required for `xwayland force scale 2` and sundry commands.

> *This turned out to be a dead end, even though it worked reasonably well, the scaling of icons was messy and it still necessitated a set of environment variables for Desktop entries*

Display management

A Reddit user `WhyNotHugo`, has an excellent line of reasoning for why a display manager is needed, rephrased to be:

- Enough system users / applications expect the existence of a `display-manager` including `systemd-logind` and `plymouth`
- No special handling needs to be done to ensure only `tty1` spawns a graphical session
- The DM is meant to ensure that window manager crashes don't yield an unlocked desktop

Anyway, the basic `agreety` should suffice for the most part; though a more pleasant setup is with `greetd-tuigreet`.

More importantly, the session itself should include the appropriate environment variables as described here and reproduced below. Essentially this involves using a runner script, `/usr/local/bin/sway-run.sh` which ensures the correct variables are setup.

```
#!/bin/sh

# Session
export XDG_SESSION_TYPE=wayland
export XDG_SESSION_DESKTOP=sway
export XDG_CURRENT_DESKTOP=sway

source /usr/local/bin/wayland_enablement.sh

systemd-cat --identifier=sway sway $@
```

Where the sourced file is:

```
#!/bin/sh
export GDK_BACKEND=wayland,x11
export MOZ_ENABLE_WAYLAND=1
export CLUTTER_BACKEND=wayland
export QT_QPA_PLATFORM=wayland-egl
export ECORE_EVAS_ENGINE=wayland-egl
export ELM_ENGINE=wayland_egl
export SDL_VIDEODRIVER=wayland
export _JAVA_AWT_WM_NONREPARENTING=1
export NO_AT_BRIDGE=1
export BEMENU_BACKEND=wayland
```

Now instead of using `tuigreet --cmd sway` in `/etc/greetd/config.toml` we can use `sway-run` as the `cmd`.

It is important to optionally provide `GDK_BACKEND` with a fallback option otherwise `zotero` and others might fail with `Error: cannot open display: :0`.

This still breaks for a bunch of special cases for which I went with a specialized `desktop` file in `$HOME/.local/share/applications/BLAH`. For example, consider the `enpass` file (since it only supports `xcb`):

```
[Desktop Entry]
Type=Application
Name=Enpass XCB
Exec=env QT_QPA_PLATFORM=xcb QT_SCALE_FACTOR=2 enpass
Icon=enpass
```

However, this is still not ideal.

XWayland proxies

Rather than work with the patched `sway` and `wlroots` packages, a recent approach by Thomas Leonard based on isolating the XWayland setup works a lot better for high DPI screens and systems. There are some caveats and still requires modified desktop files, but it works a lot better and has fewer bugs than the standard XWayland setup with `sway`. However, `xlsclients` does not track the X11 applications started in the manner described here.

```
git clone https://github.com/talex5/wayland-proxy-virtwl.git
cd wayland-proxy-virtwl
opam install .
# Installs to $HOME/.opam/default/bin/wayland-proxy-virtwl
```

For my system, the corresponding service required both a change in DPI, and the “unscaling”.

```
[Unit]
Description=Wayland-proxy-virtwl

[Service]
ExecStart=/home/$USER/.opam/default/bin/wayland-proxy-virtwl --tag="[my-vm] " --wayland-display wayland-0 --x-
display=0 --xrdb Xft.dpi:150 --x-unscale=2

[Install]
WantedBy=default.target
```

Note that the service needs the full path, not the censored one in the snippet. Now any subsequent X11 applications can be used via:

```
systemctl restart --user wayland-proxy-virtwl.service
DISPLAY=":0"  WAYLAND_DISPLAY="wayland-1" QT_QPA_PLATFORM=xcb QT_SCALE_FACTOR=1.5 enpass
DISPLAY=":0" GDK_BACKEND="x11"  WAYLAND_DISPLAY="wayland-1" gvim
```

When things work out, then the applications will have `[my-vm]` in the window title. This still requires that `xorg-xwayland` be installed, and in some situations it needs to be restarted, but by and large this is a smooth setup.

Miscellaneous

Terminal Emulators

Though `kitty` and a few other stalwarts do support `wayland`, foot seems to be the fastest, especially since it has a server client mode much like `emacs` with `foot -s` and `footclient`.

Screenshots

`slurm` and `grim` still function well together. This makes it mostly painless.

```
slurp | grim -g - - | wl-copy
```

Typically this is bound to `WIN+0` for me.

Screen Recorders

Unfortunately, the `zoom` desktop client does not support `sway` which is problematic. However, with pipewire setup, Firefox passes the gUM test for screen-sharing which is good enough. For recording stuff, OBS has a plugin, but simplescreenrecorder has a fork which works out of the box, with XWayland required for the UI though.

```
trizen -S simplescreenrecorder-wlroots-git
DISPLAY=":0"  WAYLAND_DISPLAY="wayland-1" simplescreenrecorder
```

Reference Management

I've been a `zotero` user for many years now (with WebDAV sync), however it suffers from the same UI scaling issues and cannot be fixed either by forcing `xwayland` execution nor by trying other scaling methods. The last resort is to set up a `user.js` file in `$HOME/.zotero/zotero/$PROFILE/user.js` with:

```
user_pref('layout.css.devPixelsPerPx', '2');
user_pref("extensions.zotero.fontSize", "0.3");
```

With this, the toolbar text is still too large, as are all the preference text elements, however it is still moderately usable.

Somehow this does conflict and error out with the proxied XWayland, but the regular settings work alright.

Mail Clients

The mail client story is increasingly complicated on Linux machines in general. Since Thunderbird tanked with XUL ecosystem eons ago there haven't been many contenders. Given the situation with window management, a `localserver` approach to web-clients like Mailpile [^fn:3] would have been perfect... Except it is still based around `python2` and shows no sign of moving forward. I really like Astroid, but it doesn't seem to have been updated in a while either. Thunderbird works fine too, and does not require XWayland which is a plus point.

Electron

Anything with a recent electron binary (around 13) will just need a few flags.

```
code --enable-features=UseOzonePlatform --ozone-platform=wayland
obsidian --enable-features=UseOzonePlatform --ozone-platform=wayland
```

Browsers

Personally I prefer `firefox` in-spite of its shitty tendencies on the regular editions (e.g. not allowing unsigned extensions).

Firefox

Firefox's `wayland` build on the AUR remains a nightmare to compile and work with. The developer version can leverage the `MOZ_ENABLE_WAYLAND=1` to get reasonably good results.

Chrome

As of this post the stable branch of `google-chrome` does indeed support Wayland with a switch:

```
google-chrome-stable --enable-features=UseOzonePlatform --ozone-platform=wayland
```

The stable branch is rather boring anyway, on Arch Linux atleast, the launchers support `$XDG_CONFIG_HOME/chrome-dev-flags.conf` or its equivalents.

```
--enable-features=UseOzonePlatform
--ozone-platform=wayland
```

RStudio

No `wayland` support. Needs to be forced into `xwayland` and have the fonts scaled via:

```
# For the hidpi setup
Exec=env WAYLAND_DISPLAY= QT_SCALE_FACTOR=2 /usr/bin/rstudio-bin %F
# Or with the proxied xwayland
DISPLAY=":0"  WAYLAND_DISPLAY="wayland-1" QT_SCALE_FACTOR=2 QT_QPA_PLATFORM=xcb /usr/bin/rstudio-bin
```

Conclusions

Things have definitely improved in three years, however the overall aggravation and microaggressions which come with working around window manager issues seem to be too high a price for the potential battery / quality of life savings. In particular, as before, forcing programs to run in `xwayland` makes them effectively worse than their `x11` counterparts and requires far more maintenance load. It seems likely then, that in-spite of great progress it is still probably best to stick to X11; especially for new users who might not immediately suspect the display server as a source of buggy UX elements (or screen-sharing woes).

Though the conclusions above are still valid, with the XWayland fixes, this is definitely viable and will remain my daily driver.